AQSTOQFLOW UI/UX Phase 06: Finance and Accounting Table Normalization

Date: 2026-08-28 Scope: X1 finance and accounting archetypes, first representative ledger surface only Status: Implemented with release gates pending

Decision

The first X1 slice normalizes the existing TanStack payment ledger used by the finance command center, payments, receivables, and payables views. It does not migrate unrelated tables, change finance service queries, alter posting truth, or introduce a second grid library.

Implemented

- Added `finance.payments-ledger` version 1 canonical contract with the `finance` module boundary, `finance.payments.read` permission, tenant isolation, redaction policy, bilingual labels, and server-finalized CSV/XLSX/PDF export declarations.
- Declared payment number and counterparty as frozen identity columns and amount as right-aligned numeric data. The TanStack table now applies sticky identity columns, stable row IDs, ARIA sort state, and the shared table boundary.
- Hydrated density, column visibility, and column order through the existing versioned local preference contract. Preferences are sanitized against the finance contract and cannot hide the payment identity column.
- Bound organization, period, currency, read-model, and evidence-route context at production call sites using the server-owned finance dashboard payload.
- Added finance statement export manifest and asynchronous-job receipt schemas. Matching sets at or above 1,000 rows declare `async-job`; smaller sets remain eligible for inline delivery. Server authorization and finalization remain mandatory.
- Added focused contract tests for access boundaries, frozen/numeric column semantics, bilingual multi-currency context, export mode selection, and auditable job receipts.

Preserved controls

- Finance service-owned data remains the source of business truth; this slice is presentation and contract normalization only.
- Existing RBAC, finance module entitlement, tenant scope, redaction, bilingual EN/FR formatting, and protected actions remain authoritative.
- No editable journal draft behavior was merged into posted journal/evidence tables. Journal, trial balance, P&L, cash flow, close evidence, and large-statement endpoint wiring remain subsequent X1 work.

Verification record

- Focused contract test target: `components/DataTableComponents/__tests__/FinanceAccountingContract.test.ts`.

- Existing finance payment-ledger tests remain the regression target: `components/finance/__tests__/FinancePaymentsTable.test.tsx`.
- Authenticated desktop/tablet/mobile browser evidence was not collected because approved credentials and seeded data were not supplied.
- Proposed performance budgets remain product-approval gates, not measurements or SLAs. No production promotion is claimed.

Release gates

1. Product approves the proposed X1 performance envelope before performance claims or broader finance migration.
2. A finance owner approves the period/entity/currency/evidence context and export manifest fields.
3. The server endpoint accepting export manifests supplies authorization decision IDs, idempotency, tenant checks, redaction, and asynchronous job receipts.
4. Authenticated EN/FR desktop, tablet, and mobile evidence is collected only with approved credentials and seeded data.
5. The remaining finance archetypes are migrated one at a time: receivables, payables, cash history, journals, trial balance, P&L, cash flow, and close evidence.

Residual risks

- The current finance dashboard still provides a bounded recent-payment read model; this implementation does not claim server pagination or 100,000-row browser performance.
- Sticky offsets use the payment identity column width as a stable presentation contract and require visual evidence on the approved browsers before broader adoption.
- Async export job execution is contract-ready but endpoint wiring and worker observability are not part of this narrow slice.

X1 archetype registry increment

The contract layer now also names the remaining X1 finance/accounting archetypes without migrating their existing markup: receivables, payables, cash payment history, posted journal entries, draft journal entries, trial balance, profit and loss, cash flow, and close evidence. Each contract requires period/entity context, declares frozen identity and numeric columns, preserves tenant/RBAC/module/redaction boundaries, and requires drill-to-evidence plus server-finalized exports. Close evidence is marked immutable-evidence.

Posted journal entries and editable journal drafts use distinct contract IDs. Only the draft contract declares an editing action; posted journal/evidence contracts remain read-only. Existing pages and protected actions remain service-owned and are not silently replaced by this registry.

Trial balance surface increment

The accounting trial-balance table is now the first statement page migrated onto the shared TanStack surface. Its existing `getTrialBalanceAction` remains the only data source; full-report totals remain authoritative while client filtering and pagination operate only on the returned bounded set. The surface adds contract-backed row IDs, frozen account code/name identity, right-aligned numeric debit/credit values, persisted density/order

preferences, bilingual EN/FR labels, canonical state announcements, and evidence metadata without changing accounting writes.

Verification now includes the trial-balance route suite (3 passing tests) and a direct client-surface contract assertion. The remaining statement pages are not yet migrated and remain an explicit next increment.

Financial statement detail increment

P&L and Balance Sheet account-detail sections now use the shared TanStack statement-line primitive inside their existing expandable sections. The migration preserves the `getFinancialStatementsAction` payload, period selector, posted-ledger-only semantics, balance-equation controls, and management classification information while adding French account labels, frozen code/name identity, right-aligned currency amounts, persisted density/order preferences, canonical empty-state announcements, and evidence metadata.

Journal draft/posted boundary increment

The accounting journals surface now presents two distinct canonical table scopes: editable draft entries and posted/reversed journal evidence. Draft posting remains bound to `accounting.journal.post` with fresh authentication, while posted reversal remains bound to `accounting.journal.reverse` with fresh authentication. The existing server actions, redirect behavior, and ledger-owned business truth were not moved into the table layer.

Each scope carries its own versioned contract, status-scoped canonical query, bilingual boundary locale, and explicit evidence semantics. This prevents draft editing controls from appearing in posted evidence tables and keeps reversal available only for posted rows.

Focused evidence: `app/[locale]/(dashboard)/dashboard/accounting/journals/__tests__/page.test.tsx` passed 3 tests; `components/DataTableComponents/__tests__/FinanceAccountingContract.test.ts` passed 5 tests.

X1 implementation status

The finance/accounting increment now covers the nine requested archetypes plus the close-findings companion: receivables, payables, cash history, posted journals, draft journals, trial balance, profit and loss, balance sheet, cash flow, close evidence, and close findings. Trial balance and financial-statement detail tables use the shared TanStack-based contract stack with frozen identity columns, numeric alignment, period/entity context, bilingual labels, and drill-to-evidence affordances. Close evidence remains immutable and exportable as evidence; large statement exports resolve to asynchronous job mode with a versioned manifest and job receipt contract.

Product approval remains required before treating the proposed performance budgets as commitments. Authenticated desktop/tablet/mobile evidence remains gated on approved credentials and seeded data; no unauthorised evidence was generated.

Cash and payment history normalization increment

The cash/payment history workbench now binds to the canonical `finance.cash-history` contract at its existing responsive history-shell boundary. The contract declares server-cursor ownership, tenant-scoped read access with the approved cash-drawer/payment permission alternatives, server-finalized export, and URL-safe filters for lane, location, cashier, payment method/status, cash type, effective-as-of, and date range. The canonical query

fingerprint preserves the server cursor and excludes unknown filters before the existing service action receives them.

The implementation keeps the existing server-owned hook, redaction behavior, partial-source state, responsive desktop/mobile presentation, row detail drawer, and export mutation. No client-side business truth or second data source was introduced.

Focused evidence: `components/finance/__tests__/CashPaymentHistoryCanonicalContract.test.ts` passed 2 tests and `components/finance/__tests__/CashPaymentHistoryWorkbench.test.tsx` passed 1 test.

Cash-flow statement normalization increment

The cash-flow view now renders the server-owned finance trend read model through a canonical analytical statement table instead of the prior chart-only movement presentation. The table uses the shared TanStack stack, persists versioned column visibility/order and density preferences, freezes line identity columns, right-aligns inflow/outflow/net values, carries organization, period, location, and currency context into the canonical query, and exposes the finance read-model evidence label without fabricating accounting lines.

The chart presentation remains unchanged for the other finance command-center views. Cash-flow data continues to come from the existing tenant-scoped finance dashboard service; no business calculations or accounting truth moved into the client.

Focused evidence: `components/finance/__tests__/CashFlowStatementContract.test.ts` and `components/finance/__tests__/CashFlowStatementTable.test.tsx` passed 3 tests combined.

Large-statement export queue increment

Large finance/accounting export manifests now have an executable server enqueue boundary over the existing tenant-scoped `REPORT_EXPORT` business-event outbox. The accounting export action requires `accounting.exports.create`, fresh authentication, and the accounting module entitlement; the service validates the current registered table contract, export format, source read model, redaction policy, organization context, async threshold, and actor-bound idempotency key before recording the request and returning a versioned job receipt. Status reads are tenant-scoped and map queue failure/dead-letter states to a safe failed receipt.

This increment establishes request acceptance and status semantics without treating queue acceptance as completed artifact generation. Artifact storage and worker delivery remain an operational integration gate for production rollout; no synchronous export path was silently relabeled as complete.

Focused evidence: `services/accounting/__tests__/finance-statement-export.service.test.ts` passed 5 tests.

Large-statement artifact completion increment

The asynchronous finance statement queue now has a worker-owned completion protocol. A worker must hold the tenant-scoped outbox lease before attaching a storage-provider-neutral artifact reference containing provider, opaque key, content type, byte length, SHA-256 content hash, and expiry. Completion is recorded in the existing `REPORT_EXPORT` outbox metadata and audit log before the generic outbox delivery acknowledgement releases the lease; status cannot claim `completed` without the artifact reference, and a different worker or different artifact cannot complete the same lease.

The artifact reference is intentionally not a signed download URL and does not select a storage vendor. Production wiring still needs the deployment's approved artifact store, renderer/worker, download-grant endpoint, retention policy, and monitoring. Queue acceptance and worker completion remain distinct states.

Focused evidence: `services/accounting/__tests__/finance-statement-export.service.test.ts` covers tenant/lease ownership, completed artifact receipt validation, and the unauthorized-worker rejection.

The queue also exposes an event-filtered finance worker runner. It delegates rendering and storage to an injected deployment-owned callback, rejects cross-tenant payloads, records completion through the worker lease, and lets the shared outbox gateway perform the final delivery acknowledgement. This is the integration seam for the approved production renderer/blob provider; it does not fabricate a storage implementation or a download URL.

Receivable and payable history normalization increment

The actual AR and AP history workbenches now bind to dedicated canonical contracts without relabeling payment rows as accounting documents. AR uses the posted customer receivable open-item service with stable `referenceId` identity, document/counterparty identity columns, due date/status/currency context, opening amount/open balance numeric fields, and evidence grade. AP uses the existing tenant-scoped, server-cursor supplier invoice/payment history with document/supplier identity, due date/status/currency, amount/payable movement numerics, evidence reference, supplier filters, lane filters, and cursor state.

Existing AR/AP service actions, lifecycle truth, redaction, purchasing-module enforcement, bilingual shell copy, and export mutations remain unchanged. Overview cards still use the finance dashboard read model; they are not claimed to be document-ledger rows. The dedicated contracts are the authoritative table/export context for the history surfaces.

Focused evidence: `components/DataTableComponents/__tests__/FinanceHistoryCanonicalContract.test.ts` covers AR/AP row identities, numeric columns, access boundaries, filter sanitization, and cursor preservation.

Close evidence normalization increment

The close-assurance evidence coverage surface now projects service-owned `CloseEvidenceItemDto` records through the `accounting.close-evidence` canonical contract. The projection preserves tenant-scoped period context, stable evidence/source identity, bilingual headers, availability and provenance states, evidence counts, and source hashes without changing close certification, waiver, assignment, comment, or pack-export actions. The provenance summary and evidence graph remain intentionally non-tabular views. Focused verification: `components/accounting/__tests__/CloseAssuranceCenter.test.tsx` passed with 1 suite and 6 tests.

Safe export operations readiness increment

Finance large-statement operations now expose a provider-neutral readiness snapshot and tenant-scoped queue observability without returning artifact content. The readiness contract requires deployment-owned renderer, artifact provider, download-grant policy, and retention policy declarations before reporting ready; queue metrics cover pending, locked, failed, completed, expired-artifact, and oldest-pending states from the existing `REPORT_EXPORT` outbox. The retention planner returns only expired completed opaque artifact references for a deployment-owned cleanup worker and performs no deletion itself. No storage vendor, signed content URL, artifact reader, or deletion path is fabricated. Focused verification:

`services/accounting/__tests__/finance-statement-export-operations.service.test.ts` passed with 1 suite and 3 tests.

The non-content operations snapshot is exposed through `getFinanceStatementExportOperationsSnapshotAction` under `accounting.reports.read`, handler-derived tenant scope, and enforced accounting module access. It is intentionally operational metadata only; it does not expose artifact contents or bypass the deployment-owned storage boundary.

Focused action-boundary verification: `actions/accounting/__tests__/finance-statement-export.actions.test.ts` and `services/accounting/__tests__/finance-statement-export-operations.service.test.ts` passed together with 2 suites and 5 tests, covering authenticated tenant derivation, permission/module configuration, queue metrics, retention-candidate filtering, and non-secret environment configuration resolution. Readiness uses `STOQUIFY_FINANCE_EXPORT_RENDERER_ID`, `STOQUIFY_FINANCE_EXPORT_ARTIFACT_PROVIDER`, `STOQUIFY_FINANCE_EXPORT_ARTIFACT_APPROVAL_REFERENCE`, `STOQUIFY_FINANCE_EXPORT_DOWNLOAD_GRANT_POLICY`, and `STOQUIFY_FINANCE_EXPORT_RETENTION_POLICY`; absent or blank values remain not ready, and the artifact store cannot report configured without both a provider identifier and approval reference.

Scale-trial linkage

The shared scale contract now identifies `accounting.trial-balance` as the single future analytical desktop virtualization surface. This selects the X1 candidate without enabling virtualization, transferring unbounded rows to the browser, or claiming measured performance, accessibility, print/export, zoom, or screen-reader outcomes. Focused verification: `components/DataTableComponents/__tests__/NextPhaseContracts.test.ts` passed with 1 suite and 5 tests.

X1 evidence intake binding (not certification)

The X1 finance/accounting route family is now bound to the implemented canonical contracts, finance pilots, and focused contract-test artifacts through `components/DataTableComponents/finance-x1-evidence-binding.ts`. The binding requires authenticated EN/FR evidence at desktop, tablet, and mobile viewports, but remains `blocked-awaiting-approval` until approved credentials, approved seeded data, and product-approved performance budgets are available.

The binding deliberately contains no screenshot paths, measurements, production-budget approval, provider-specific artifact references, or financial artifact content. It is a metadata-only traceability gate and must not be treated as certification or as evidence that the proposed budgets are current measurements or SLAs.

Local verification

The metadata-only X1 evidence binding contract passed its focused test: 1 suite and 3 tests. This verifies traceability and approval-gate invariants only; it does not constitute authenticated desktop/tablet/mobile evidence, product budget approval, or finance artifact certification.

Shared-stack integration

`financeX1EvidenceBinding` is exported through the existing `components/DataTableComponents/index.ts` barrel. This keeps the X1 traceability contract discoverable from the shared table stack without introducing a second grid, a provider-specific download path, or any new access to financial artifact contents.

Post-integration verification

After exposing the binding through the shared `DataTableComponents` barrel, the focused X1 binding test again passed: 1 suite and 3 tests. The result covers metadata and approval-gate invariants only; it does not certify production performance, authenticated localization, or finance artifact delivery.

Focused X1 contract-suite verification

The focused X1 contract suite passed with 5 suites and 19 tests: scale-contract invariants, close-evidence table behavior, finance export operations, protected export action boundaries, and the evidence-binding gate. This remains local verification only and does not replace approved authenticated EN/FR desktop, tablet, and mobile evidence.

Barrel-export verification

The focused X1 evidence-binding test now imports through `components/DataTableComponents/index.ts`, confirming that the traceability contract is exposed by the canonical shared table stack. The test passed with 1 suite and 3 tests. No certification claim is made without approved budgets and authenticated EN/FR seeded-data evidence.